

Beyond Single-Granularity Prompts: A Multi-Scale Chain-of-Thought Prompt Learning for Graph

Ziyu Zheng

School of Computer Science and
Technology,
Xidian University,
Xi'an, China
zhengziyu@stu.xidian.edu.cn

Yaming Yang

School of Computer Science and
Technology,
Xidian University,
Xi'an, China
yym@xidian.edu.cn

Ziyu Guan

School of Computer Science and
Technology,
Xidian University,
Xi'an, China
zyguan@xidian.edu.cn

Wei Zhao*

School of Computer Science and
Technology,
Xidian University,
Xi'an, China
ywwzhao@mail.xidian.edu.cn

Xinyan Huang

School of Artificial Intelligence,
Xidian University,
Xi'an, China
xinyanh@stu.xidian.edu.cn

Weigang Lu

Department of Civil and
Environmental Engineering,
The Hong Kong University of Science
and Technology,
Hong Kong SAR
weiganglu314@outlook.com

Abstract

The “pre-train, prompt” paradigm, designed to bridge the gap between pre-training tasks and downstream objectives, has been extended from the NLP domain to the graph domain and has achieved remarkable progress. Current mainstream graph prompt-tuning methods modify input or output features using learnable prompt vectors. However, existing approaches are confined to single-granularity (e.g., node-level or subgraph-level) during prompt generation, overlooking the inherently multi-scale structural information in graph data, which limits the diversity of prompt semantics. To address this issue, we pioneer the integration of multi-scale information into graph prompt and propose a Multi-Scale Graph Chain-of-Thought (MSGCOT) prompting framework. Specifically, we design a lightweight, low-rank coarsening network to efficiently capture multi-scale structural features as hierarchical basis vectors for prompt generation. Subsequently, mimicking human cognition from coarse-to-fine granularity, we dynamically integrate multi-scale information at each reasoning step, forming a progressive coarse-to-fine prompt chain. Extensive experiments on eight benchmark datasets demonstrate that MSGCOT outperforms the state-of-the-art single-granularity graph prompt-tuning method, particularly in few-shot scenarios, showcasing superior performance. The code is available at: <https://github.com/zhengziyu77/MSGCOT>.

CCS Concepts

• Information systems → Web mining; • Computing methodologies → Unsupervised learning.

*Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792115>

Keywords

Graph Neural Networks; Graph Prompt Learning; Few-Shot Learning

ACM Reference Format:

Ziyu Zheng, Yaming Yang, Ziyu Guan, Wei Zhao, Xinyan Huang, and Weigang Lu. 2026. Beyond Single-Granularity Prompts: A Multi-Scale Chain-of-Thought Prompt Learning for Graph. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792115>

1 Introduction

Graph Neural Networks (GNNs) have been widely adopted in real-world scenarios, such as social networks [18], anomaly detection [16], and recommendation systems [3], due to their ability to capture complex structural dependencies among data. In recent years, the scarcity of labels in practical settings has spurred extensive research on the “pre-training and fine-tuning” paradigm for GNNs [15, 31, 33, 37]. This paradigm utilizes self-supervised learning to derive generic, task-agnostic representations from unlabeled graphs, followed by fine-tuning the pre-trained model on downstream tasks using task-specific labels. However, these approaches suffer from an inherent limitation: the discrepancy between pre-training objectives and downstream tasks, which leads to suboptimal performance [26].

To address this issue, recent work introduces prompt tuning as an alternative to fine-tuning. Prompt tuning is initially proposed in NLP and achieving remarkable success [13], and can effectively bridge the gap between pre-training objectives and downstream tasks by incorporating lightweight learnable prompt parameters to modify inputs or outputs, without updating the parameters of the pre-trained model. Inspired by this, pre-training and prompt tuning have been extended to the graph domain [4, 24, 25]. Compared to fine-tuning, graph prompt exhibits lower computational overhead and better generalization.

Existing graph prompt learning methods can be categorized into two categories based on dependency on pre-training strategies. Pre-training-dependent methods [2, 14, 25] design unified templates for pre-training objectives and downstream tasks to reduce their discrepancy. Pre-training-agnostic methods [4, 6, 12, 35] enhance generality by inserting learnable feature prompt vectors into inputs or outputs, requiring no specific pre-training strategy. Further, we classify pre-training-agnostic methods by their prompting mechanisms: (1) Single-step prompt tuning [4, 6, 12, 34] employ a direct generation approach where the complete prompt for downstream tasks is produced in a single forward pass. (2) Multi-step prompt tuning [35] progressively derives final prompts through iterative inference, mimicking the Chain-of-Thought (CoT) [29] rationale to achieve stronger expressiveness.

However, these feature-based prompt tuning methods focus exclusively on single granularity (node-level, edge-level, or subgraph-level) during prompt generation, overlooking the intrinsically coexisting multi-scale information from nodes to hierarchical subgraphs in real-world graphs. As illustrated in Figure 1, prior works synthesize node/subgraph-specific prompt features using learnable basis vectors or a condition network. GCOT is the current state-of-the-art method designed with text-free multi-step refinement prompts for graph chain of thought architectures, but it still remains confined to node-level granularity at each step. This means that only a single perspective is considered in the prompt generation, fundamentally limiting prompt diversity.

To overcome the singularity of prompt granularity in existing methods, in this work, we propose a Multi-Scale Graph Chain-of-Thought (MSGCOT) prompt tuning framework. Achieving this design requires solving two core challenges.

First, how to construct multi-granularity information for prompt generation? In hierarchical graph representation learning [7], the original graph is progressively partitioned into smaller subgraphs [9, 32] by merging nodes or edges to extract multi-scale features. However, these methods are typically designed under supervised settings and ultimately aim to learn a graph representation for solving graph-level tasks. In graph prompt learning, our objective is to obtain a general multi-scale representation that can be adapted to various downstream tasks. Therefore, instead of directly applying the multi-scale representation to downstream tasks, we treat them as an intermediate basis vector pool to enrich node-level prompts with multi-scale features. Specifically, we design a lightweight coarsening network with a low-rank matrix architecture to learn hierarchical coarsened representations. These representations serve as a multi-scale basis vector pool for generating node feature prompts infused with diverse structural granularities.

Second, how to integrate multi-scale information into multi-step reasoning? While the multi-scale basis vector pool provides rich hierarchical features, direct aggregation of all granularities in a single step could cause feature interference and suboptimal prompt generation. Inspired by Chain-of-Thought (CoT) approaches [5, 29], we find that multi-step progressive prompt optimization is a superior solution. In the NLP domain, CoT requires manually designed textual templates to guide the model’s reasoning. For non-textual graph data, we innovatively treat hierarchical coarsened representations as structured thought—from global topology to local

details. These multi-granularity representations serve as progressively detailed “textual examples” in CoT, thereby replacing the guiding role of text. We further propose a backtracking-based progressive prompt optimization strategy: at each reasoning step, the pre-trained embedding is iteratively refined by integrating features of specific granularities, achieving a coarse-to-fine reasoning chain. This process mirrors human cognitive refinement, similar to NLP’s iterative text refinement for enhanced answer accuracy. The final generated prompts not only retain the structural outline of coarse granularity but also incorporate discriminative features from fine granularity, enabling more precise capture of hierarchical semantics compared to traditional single-granularity methods.

The main contributions of this work are summarized as follows:

- We propose the first graph chain of thought framework that integrates multi-granularity information, overcoming the single-granularity limitation of existing methods.
- We simulate human cognition from coarse-to-fine granularity by designing a low-rank coarsening network for multi-scale feature extraction and a backtracking prompt mechanism for progressive prompt generation.
- Extensive experiments on eight benchmark datasets for node and graph classification demonstrate that our multi-step, multi-granularity prompting framework outperforms state-of-the-art single-granularity methods.

2 Related Work

2.1 Graph Pre-training

In recent years, graph pre-training have attracted extensive research due to their ability to operate without labeled data [15, 36]. These methods are primarily categorized into contrastive learning [28, 33] and generative learning [8, 37]. Contrastive learning methods construct multiple views by sampling positive and negative samples, then maximizing the consistency between positive samples. Generative learning methods pre-train encoders by reconstructing node features or graph structures. These approaches employ different pre-training objectives to transfer pre-trained knowledge to various downstream tasks. However, due to the significant gap between downstream tasks and pre-training objectives [25, 26], the performance on downstream tasks may be compromised.

2.2 Graph Prompt-Tuning

Recent advances in graph prompt learning have sought to bridge the gap between pre-training and downstream tasks by introducing task-specific prompts. Early works, such as GPPT [24] and Graph-Prompt [14], reformulate downstream tasks as link prediction or subgraph similarity tasks. All-in-One [25] unified various downstream tasks as graph-level tasks. Feature-based prompt method GPF+ [4] and SUPT [12] introduced feature-space prompting and subgraph prompt, where learnable vectors modify input. Edge-Prompt [6] inserts edge-level prompts in the aggregation at each level from an edge perspective. ProNOC [34] and DAGPrompt [2] accounted for node heterophily in the prompt by preserving neighbourhood similarity. The aforementioned methods all belong to single-step prompt tuning approaches. GCOT [35] introduces the chain of thought to generate multi-step prompts. However, these methods only focus on single-granularity information during prompt

	Node Prompt	Subgraph Prompt	Condition Prompt	Chain Prompt	Ours
Prompt Generation					
Prompt Step	Single-Step	Single-Step	Single-Step	Multi-Step	Multi-Step
Prompt Scale	Single-Scale	Single-Scale	Single-Scale	Single-Scale	Multi-Scale

Figure 1: The comparison of graph prompt tuning methods in the existing studies.

generation, ignoring the rich multi-granularity features that co-exist in real-world graphs.

2.3 Graph Coarsening

Graph coarsening generates simplified graphs by merging nodes or edges. Hierarchical graph coarsening techniques sample a hierarchical learning structure to obtain multiple subgraphs at different scales. Traditional methods compress the graph through spectral clustering [19], non-negative matrix factorisation [30]. In recent years, learnable graph coarsening techniques [7, 32] progressively aggregate the nodes into coarser graphs through learnable assignment matrices, while preserving important structures, and ultimately generating compact graph-level representations. Graph coarsening naturally provides a hierarchical abstraction mechanism for us to capture multi-scale information in graph prompt learning and learn multi-scale prompts.

3 Preliminaries

Notions. We denote a graph as $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ represents the set of the nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ represents the edge set. Let N represent the total number of nodes. Node features are usually represented by the matrix $\mathbf{X} \in \mathbb{R}^{N \times F}$, where each row $\mathbf{x}_i$ corresponds to the node i 's F -dimensional feature vector. Let $\mathbf{A} \in \mathbb{R}^{N \times N}$ denote the adjacency matrix. Its element $A_{ij} = 1$ if there exists an edge between node i and j , and otherwise, $A_{ij} = 0$.

Graph Neural Networks. Most Graph Neural Networks operate under a message-passing paradigm [10]. Within this scheme, the representation $\mathbf{h}_i$ of each node i is updated iteratively in each layer. This update involves collecting messages from the node's neighborhood, denoted $\mathcal{N}(i)$. Specifically, the representation $\mathbf{h}_i^l$ at layer l is computed by first applying an aggregation function $\text{AGGR}(\cdot)$ to the representations from layer $(l-1)$ within $\mathcal{N}(i)$, followed by an $\text{UPDATE}(\cdot)$ function:

$$\tilde{\mathbf{h}}_i^l = \text{AGGR}^{(l)}(\{\mathbf{h}_j^{l-1} : j \in \mathcal{N}(i)\}), \quad (1)$$

$$\mathbf{h}_i^l = \text{UPDATE}^{(l)}(\tilde{\mathbf{h}}_i^l, \mathbf{h}_i^{l-1}). \quad (2)$$

In the subsequent sections, to simplify the expression, we denote the encoding process of GNN as follows:

$$\mathbf{H} = \text{GNN}(\mathbf{X}, \mathbf{A}), \quad (3)$$

where $\mathbf{H}$ denotes the final embedding after encoding, and $\text{GNN}(\cdot)$ can conform to any encoder of the message passing paradigm, including but not limited to GCN, GAT, GraphSAGE, and other variants that share this fundamental computational framework.

4 Proposed Method

4.1 Overall Framework

Our objective is to develop a multi-step graph prompt framework that incorporates multi-scale information, as illustrated in Figure 2. Specifically, in each reasoning step, the graph coarsening network constructs multi-scale coarsened representations that encompass both coarse-grained and fine-grained information. These coarsened representations as thoughts form a hierarchical basis vector pool for prompt generation. Subsequently, a backtracking-based prompt mechanism is employed to incrementally integrate prompts from coarse to fine granularity, generating hierarchical prompts. Finally, the refined representation is optimized via downstream task loss.

4.2 Multi-scale Prompt

Node Level Prompt. During the prompt-tuning phase, the parameters of the pre-trained model remain frozen. Using the frozen embeddings directly as input to the coarsening network would limit the model's ability to adapt to downstream tasks. To ensure that the input can be adjusted during training while preserving hierarchical information when extracting multi-scale features, we first generate node-specific prompts for each node. Similar to GCOT [35], we employ a conditional network along with the initial pre-trained embeddings to produce node-level prompt vectors $\mathbf{P}_x$, which serve as task-specific multi-scale features.

$$\mathbf{P}_x = \text{CONDNET}(\mathbf{H}), \quad (4)$$

where CONDNET is a conditional network by the parameter ϕ . The conditional network can be viewed as a hypernetwork [38], i.e., a lightweight auxiliary network. To enhance task-relevant features

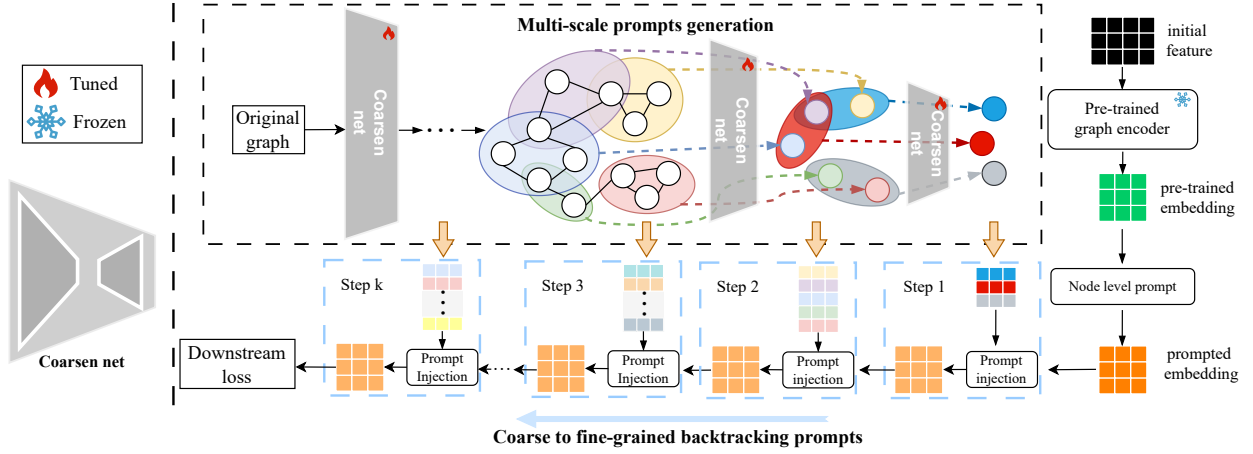


Figure 2: The framework of the proposed MSGCOT.

while suppressing irrelevant noise in the pre-trained representations. Then we use the prompt to modify the original features and obtain prompted embeddings via a pre-trained encoder:

$$\hat{\mathbf{H}} = \text{GNN}(\mathbf{X} \odot \mathbf{P}_x, \mathbf{A}), \quad (5)$$

The node representation after prompting indicates that it contains task adaptation information. The prompted embedding serves a dual role: as an augmented input to the coarsening network and as the initial state of the inference chain.

Multi-scale Thought Construction. To inject multiple levels of information into the prompting process, rather than relying only on node iteration updates. We are incrementally updating prompts by constructing multi-scale thoughts so that the prompts contain information at multiple granularities. In traditional CoT [29], thought is defined as the instruction under each reasoning step. Here, we use coarsened representations at different scales to analogise thought under different stages.

For thought construction, we need to generate a node assignment matrix for each scale. Regarding the implementation of the assignment matrix, there are parametric and non-parametric approaches in earlier work [9, 32]. In this work, we design a lightweight coarsening network to implement a soft coarsening strategy to generate the assignment matrix $\mathbf{S}^l$:

$$\mathbf{S}^l = \text{Softmax}(W_{up}^l (\sigma(W_{down}^l \mathbf{T}^{l-1}))), \mathbf{T}^0 = \hat{\mathbf{H}}, \quad (6)$$

where the coarsened network consists of a $W_{down} \in \mathbb{R}^{d \times r}$ and $W_{up} \in \mathbb{R}^{r \times C^l}$ ($r \ll d$). σ is the ELU activation function. Here we implement it using the idea of low-rank decomposition [11]. The number of parameters of the improved coarsening network is much smaller than using an MLP directly. It's consistent with the low resource requirement in prompt learning. After obtaining the assignment matrix, we compute the node representations under coarser granularity:

$$\mathbf{T}^l = \mathbf{S}^{lT} \mathbf{T}^{l-1}, \quad (7)$$

where $\mathbf{T}^l \in \mathbb{R}^{n^l \times d}$ denotes the coarsened node representation. Repeating the above operation several times, we obtain a hierarchical

thought pool with different levels. These representations serve as base vectors representing different granularities to guide the generation of multi-scale prompts. Instead of using randomly initialised basis vectors directly in earlier methods [4, 6, 12], here we construct basis vectors incorporating specific scale information.

$$\mathbf{T} = [\mathbf{T}^l, \mathbf{T}^{l-1}, \dots, \mathbf{T}^1], \quad (8)$$

The hierarchical thought pool contains diverse levels and granularities of information, and pre-learned multi-scale thoughts can be used as additional instructions for each reasoning step to generate prompts related to different granularities of information.

Coarse to Fine-grained Backtracking Prompts. CoT breaks down a complex problem into multiple steps and obtains a more accurate answer by progressively refining the manually designed text. Although GCOT also adopts multi-step prompt generation, the generation of prompts at each step relies on the optimization of the node's features, which is limited to a single granularity. This resembles repeatedly adjusting the same feature dimensions without introducing new perspectives, ultimately constraining prompt semantic diversity. To overcome this, we propose a granularity-aware backtracking mechanism that generates prompts step by step from coarse-grained to fine-grained using multi-granularity basis vectors. This simulates the progressive refinement of textual inputs in large-language-model Q&A. For each step, the prompts are generated as follows:

$$\mathbf{p}_i^{l+1} = \sum_j \alpha_{ij}^{l+1} \mathbf{t}_j^l, \alpha_{ij}^{l+1} = \frac{\exp(\mathbf{t}_j^l \hat{\mathbf{h}}_i^l)}{\sum_k \exp(\mathbf{t}_k^l \hat{\mathbf{h}}_i^l)}, \quad (9)$$

where $\mathbf{p}_i^{l+1}$ denotes the node prompts computed from the thoughts in the hierarchical thought pool, α_{ij}^{l+1} computes the attention coefficient of the node-level prompts concerning the coarse-grained thoughts, and $\hat{\mathbf{h}}_i^l$ denotes the vector of prompts for the previous step of node i , where $\hat{\mathbf{h}}_i^0 = \hat{\mathbf{h}}_i$. Next, we inject the prompts with multi-scale information to update the node representation:

$$\hat{\mathbf{h}}_i^{l+1} = \hat{\mathbf{h}}_i^l + \mathbf{p}_i^{l+1}, \quad (10)$$

Based on the constructed multi-layer thought pool, we can use its multi-granularity information to update the representation incrementally. However, the addition of multi-level coarse-grained information may cause the node's unique information to be lost; therefore, we introduce an external cosine reconstruction loss [8] to constrain the consistency of the node representations with the pre-trained embeddings after prompting.

$$\mathcal{L}_r = \sum_i \frac{1}{N} \left(1 - \frac{\hat{\mathbf{h}}_i \cdot \mathbf{h}_i}{\|\hat{\mathbf{h}}_i\| \cdot \|\mathbf{h}_i\|}\right)^\gamma, \gamma \geq 1, \quad (11)$$

where γ denotes the scaling factor employed to adjust feature reconstruction weights, with parameter settings adhering to prior work. This loss constrains node representations, preventing node-level information loss arising from the introduction of coarse-grained prompts following multi-level prompting.

4.3 Prompt Tuning

In the prompt tuning phase, we follow the formulation of [14], which defines the task within a subgraph similarity framework. Accordingly, our loss function remains consistent with this design and is specified as follows:

$$\mathcal{L}_{ds} = - \sum_{l=0}^L \sum_{(x_i, y_i) \in \mathcal{D}^{\text{train}}} \ln \frac{\exp(\text{sim}(\hat{\mathbf{h}}_{x_i}, \hat{\mathbf{h}}_{y_i})/\tau)}{\sum_{c \in \mathcal{Y}} \exp(\text{sim}(\hat{\mathbf{h}}_{x_i}, \hat{\mathbf{h}}_c)/\tau)}, \quad (12)$$

where τ denotes the temperature constant and $\text{sim}(\cdot)$ denotes the cosine similarity. $\hat{\mathbf{h}}_{x_i}$ denotes the final node embedding or graph embedding, and $\hat{\mathbf{h}}_c$ the prototype embedding for class c , obtained from the class mean of the labelled nodes or graphs of that class. Thus, the final loss is:

$$\mathcal{L}_{\text{final}} = \mathcal{L}_{ds} + \alpha \mathcal{L}_r, \quad (13)$$

where α denotes the weight of the reconstruction loss. For the node classification task, it is important to maintain the original node-level information while injecting multi-granularity information, while for graph classification, the focus is more on coarse-grained graph-level information. During prompt tuning, only the weights of the lightweight coarsened network and the weights of the node-level conditional network are updated.

4.4 Complexity Analysis

The computational complexity of MSGCOT primarily stems from two key components: multi-scale information extraction and a chain of graph prompt generation. Let N denote the total number of nodes, C^l represent the coarsened nodes at layer l , L be the number of coarsening layers, d be the hidden dimension, and r be the low-rank dimension. For the coarsening network, each layer's time complexity is $O(C^{l-1}(dr + rC^l))$ where $C^0 = N$. With a coarsening ratio $c = C^l/C^{l-1} < 1$ ($C^l = c^l N$), the total complexity across L layers becomes $\sum_{l=1}^L O(c^{l-1}N(dr + rC^l))$, which by geometric series convergence simplifies to $O\left(\frac{dr(1-c^L)N}{1-c} + \frac{rc(1-c^{2L})N^2}{1-c^2}\right)$. The prompt generation phase contributes $\sum_{l=1}^L O(N^2 c^l d) = O\left(\frac{N^2 c(1-c^L)d}{1-c}\right)$. Experimentally, setting $c \in [0, 0.2]$ makes c^L and c^{2L} negligible, reducing the total complexity to $O\left(\frac{drN}{1-c} + \frac{rcN^2}{1-c^2} + \frac{dcN^2}{1-c}\right)$. For small c (≤ 0.2), the denominators $(1-c)$ and $(1-c^2)$ approach 1, further

simplifying to $O(drN + (r+d)cN^2)$. When $cN \ll N$, the overall complexity becomes nearly linear, ensuring computational feasibility and efficiency. Please see the Sections 5.7 for more details about time-efficiency and parametric experiments.

5 Experiments

5.1 Experimental Setting

Datasets. We evaluate the performance of MSGCOT using several datasets from different domains, including citation networks, e-commerce, protein structures, and molecular graphs. For the node classification task, we used three citation network datasets, Cora [17], Citeseer, Pubmed [21], and the e-commerce co-purchase network Photo [22]. For graph classification, we used a protein structure dataset PROTEINS [1], a molecular graph dataset MUTAG, BZR, and COX2 [20].

Baselines. We compare MSGCOT with current state-of-the-art methods. These baselines fall into three categories: (1) Supervised learning. We use a supervised approach on GCN [10], GAT [27] to train on downstream labels. (2) Pre-training + fine-tuning. LP [36], GraphCL [33], and DGI/InfoGraph [23, 28] employ a pre-training and fine-tuning strategy, where the model is pre-trained on unlabeled data and then fine-tuned on downstream tasks. (3) Pre-training + Prompt: We uniformly use the link prediction task to pre-train the GNNs, and use graph prompt methods for prompting, including single-step prompt methods All-in-One [25], GPF, GPF+ [4], SUPT [12], GraphPrompt [14], EdgePrompt, EdgePrompt+ [6], DAG-Prompt [2], and multi-step prompt method GCOT [35].

Implementation details. We evaluate our approach on a few-shot node classification and graph classification tasks. For each class, we randomly select k nodes or graphs as the training set. For node classification, following the GCOT setup, we randomly sample 1,000 nodes from the remaining nodes as the test set. For graph classification, we use the remaining samples as the test set. We report the model accuracy. To ensure a fair comparison, all prompt-based models adopt link prediction as the pre-training strategy, using GCN as the backbone with a hidden layer size of 256. The number of coarsening layers is set to 2, and the coarsening ratio is selected from 0.01, 0.1, 0.2, 0.3. The hidden dimension of the low-rank decomposition matrix is set to 8. The prompt constraint weight α is set to 1 for node classification and 0 for graph classification. We conduct 100 random sampling trials, generating one-shot tasks for both node classification and graph classification, and report the mean and variance of the results. The pre-training strategies and hyperparameter settings for other baseline methods follow their original papers. All experiments are conducted on an NVIDIA RTX 4090D GPU with 24GB of memory. Experimental results using GraphCL as a pre-training task are provided in Appendix B.1. Detailed parameter settings can be found in Appendix A.2.

5.2 Performance on One-Shot Classification

We present the results of node classification and graph classification in Table 1, revealing the following findings: (1) MSGCOT consistently outperforms all baseline methods across all settings. On the Cora dataset, MSGCOT achieves a 3.68% improvement over

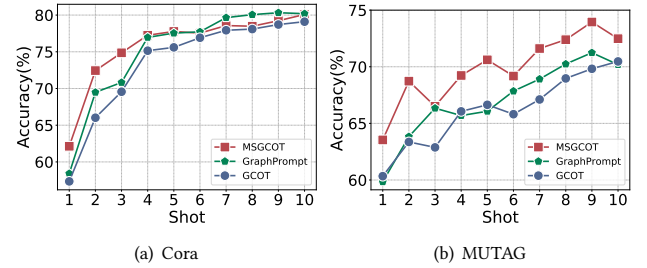
Table 1: Performance comparison on node and graph classification tasks. The best and second-best results are highlighted in bold and underlined, respectively.

	Node Classification				Graph Classification			
	Cora	Citeseer	Pubmed	Photo	MUTAG	COX2	BZR	PROTEINS
GCN	33.33±13.68	26.28±8.79	52.26±8.95	60.79±11.96	50.94±7.45	50.05±4.79	49.98±5.85	52.38±6.88
GAT	31.24±15.05	29.20±8.85	47.62±9.40	50.74±13.78	48.87±8.54	49.23±7.40	50.37±8.40	53.41±8.56
LP	56.51±13.48	43.52±9.34	53.98±7.86	62.43±9.32	59.13±14.54	51.28±4.56	51.37±9.06	52.38±5.91
DGI/Infograph	55.69±12.32	45.64±9.32	54.38±10.26	63.98±9.60	58.92±16.34	51.58±14.68	51.28±9.84	53.64±8.02
GraphCL	55.35±10.72	45.64±10.03	53.54±8.47	64.56±9.85	59.02±14.13	51.64±14.37	51.53±13.58	54.23±7.65
All-in-One	51.74±12.54	42.39±8.27	62.93±10.75	66.06±10.63	58.58±14.25	53.33±14.12	48.95±13.68	53.52±8.12
GPF	58.45±13.35	46.22±8.51	<u>63.40±9.80</u>	<u>67.37±10.42</u>	<u>62.11±14.95</u>	53.96±14.40	48.27±12.76	55.31±8.96
GPF+	57.07±14.14	44.03±8.74	56.87±10.88	65.77±10.54	58.63±14.92	54.60±13.57	50.73±13.66	54.44±9.08
SUPT	55.70±13.55	43.85±8.65	57.86±11.31	63.45±10.30	60.31±16.44	54.88±13.53	48.38±10.41	54.36±8.96
GraphPrompt	58.42±13.31	46.09±8.42	63.31±9.84	67.35±10.38	59.84±14.66	51.44±4.30	51.27±8.92	53.44±5.99
DAGPrompt	57.77±12.56	46.23±4.46	60.51±10.80	54.73±10.83	58.99±14.69	55.00±13.00	<u>55.49±13.74</u>	<u>56.22±10.53</u>
EdgePrompt	57.14±14.21	44.01±8.62	53.66±11.86	64.90±10.83	62.06±14.40	53.83±14.66	46.94±12.47	55.23±9.90
EdgePrompt+	55.93±14.24	44.05±8.93	57.59±11.15	64.50±10.08	59.74±15.45	<u>55.37±13.79</u>	48.66±12.03	54.93±9.36
GCOT	<u>59.54±13.60</u>	<u>48.13±8.89</u>	63.38±9.98	66.98±10.65	60.34±14.70	52.09±12.34	54.45±16.19	55.07±10.51
MSGCOT	62.13±17.53	49.05±11.41	64.67±10.41	68.01±10.39	63.54±14.94	73.62±6.43	69.85±11.65	57.83±2.71

the second-best method GPF, and demonstrates significant gains compared to the multi-step prompting approach GCOT, highlighting the importance of hierarchical prompts for node classification tasks. (2) For graph classification, MSGCOT achieves substantial improvements of 5-20% over the strongest baselines. Notably, on the COX2 dataset, it surpasses the second-best method DAGPrompt by 18.62%, as the multi-scale prompts effectively capture latent substructures. Unlike GCOT, which only incorporates node-level information in multi-step prompts, MSGCOT dynamically adjusts prompt granularity to preserve local structures while enhancing global representations, demonstrating superior performance.

5.3 Performance on Few-Shot Classification

To explore the impact of the amount of labelled data on the model performance, we adjust the number of shots. As shown in Figure 3, we find that MSGCOT exhibits significant advantages in the few-shot scenario. In the node classification, MSGCOT outperforms the baseline methods by an average of 5-8% in the 1-3 shot setting, and as the sample size increases to 5-10 shots, the performance of all methods improves, but MSGCOT remains competitive. In the graph classification, MSGCOT demonstrates significant advantages at different shots on MUTAG, confirming the robustness of the method to data scarcity. When the sample size is increased, the performance remains superior compared to the multi-step single-granularity prompt method GCOT. These results validate the effectiveness of the multi-scale prompt in few-shot learning.

**Figure 3: Impact of shots number analysis.**

5.4 Ablation study

To comprehensively evaluate the effectiveness of the multi-scale prompt mechanism in the MSGCOT framework, we conducted systematic ablation studies by comparing MSGCOT with four variants: (1) w/o multi-scale prompt(MSP): removing multi-scale prompts and using only node-level prompts; (2) w/o reconstruction loss(RE): eliminating the reconstruction constraint on multi-level node prompts; (3) w/o traceback (TB): employing a unidirectional process (fine-to-coarse) for prompt addition; and (4) w/o incremental update (IU): retaining only the final prompt.

As shown in Table 2: (1) Multi-scale prompts are crucial for performance. Their removal reduces node classification accuracy by 5.52% on average and leads to a more pronounced drop in graph

classification (17.7% on average), validating their critical role in capturing hierarchical structural features. (2) The reconstruction loss significantly impacts node classification, while graph classification relies more on global information. (3) The backtracking mechanism is particularly vital for graph classification; performance drops sharply by 12–15% when using unidirectional prompts, confirming the importance of dynamically adjusting prompt granularity. (4) Progressive prompt updates yield a 2–5% performance gain, indicating that intermediate-layer prompts carry complementary hierarchical information. These results demonstrate that MSGCOT effectively coordinates features across different granularities, verifying the efficacy of its multi-level, multi-scale prompt design.

Table 2: Performance comparison of different variants

	Cora	Citeseer	COX2	MUTAG
w/o MSP	56.61±14.66	43.66±9.03	51.49±4.21	60.13±14.69
w/o RE	46.98±15.90	39.40±10.72	-	-
w/o TB	59.67±17.01	48.38±10.51	56.33±12.83	62.45±16.27
w/o IU	60.61±16.45	46.59±10.00	53.58±14.59	62.10±14.57
FULL	62.13±17.53	49.05±11.41	73.62±6.43	63.54±14.94

5.5 Hyperparameter Analysis

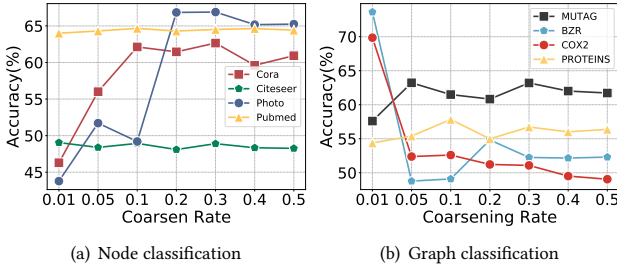


Figure 4: The sensitivity of the coarsening rate.

Coarsening rate. We control the number of coarsened nodes via the coarsening rate, where lower values emphasize coarse-grained information. As shown in Figure 4, node-level tasks peak at moderate rates (0.1–0.3), as excessively low rates lose fine-grained details. Graph classification remains robust across a wider range (0.05–0.3), demonstrating stronger robustness to coarsening.

Coarsening layer. The number of coarsening layers defines both the depth of multi-scale prompting and the reasoning steps. At each step, hierarchical prompts of varying granularity are integrated. We analyze the effect of layer count in Figure 5 and observe: (1) For node-level tasks, performance initially improves then declines with more layers. On Cora, two layers achieve an optimal balance between node semantics and subgraph structure; excessive layers induce over-smoothing. (2) For graph-level tasks, performance steadily improves with deeper hierarchies. On PROTEINS,

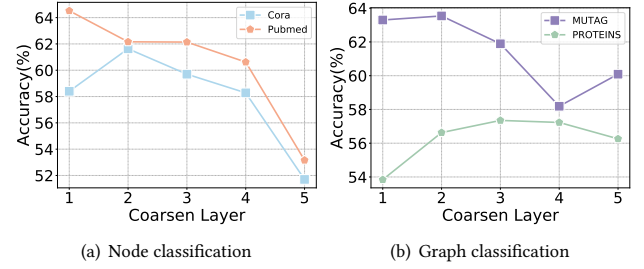


Figure 5: The sensitivity of the coarsening layer number.

performance remains strong at 3–4 layers, indicating enhanced benefits from hierarchical feature extraction. This confirms graph tasks’ greater reliance on multi-scale structures and validates our hierarchical prompting mechanism.

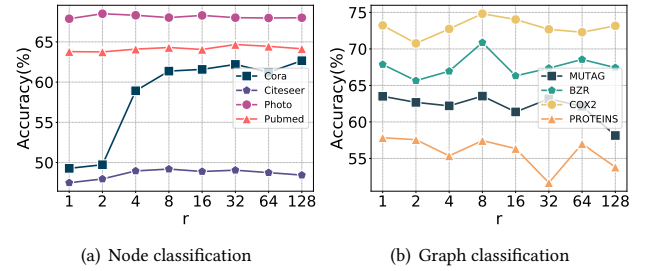


Figure 6: The sensitivity of the hidden dimensions of the coarsening net.

Hidden dimensionality of coarsening net. We design an efficient coarsening network to capture multi-scale information and improve generalization by controlling parameter scale. To assess its efficacy, we evaluate performance across different hidden dimensions r (Figure 6). For node classification, accuracy stabilizes when $r > 8$, with peak results at $r = 8$. Graph classification exhibits greater robustness to dimension changes, achieving strong performance even at $r = 1$. A slight overfitting emerges in node tasks when $r > 32$. These results confirm that MSGCOT’s low-rank design effectively balances efficiency and performance stability, sustaining strong generalization with minimal dimensionality.

5.6 The Effectiveness of Multi-Scale Prompts

To validate the effectiveness of multi-scale prompts, we evaluate a non-parametric variant (MSGCOT-P) that replaces the trainable coarsening network with precomputed multi-scale partitions $[S^1, S^2, \dots, S^L]$ derived from graph coarsening algorithms [9]. This eliminates the need for learnable assignment matrices, reducing trainable parameters to only the node-level prompt components. Therefore, this version has smaller training parameters. As shown in Figures 7(a) and 7(b), MSGCOT-P achieves comparable or superior results to GCOT across all datasets, with notable improvements on Photo (+1.69%) and MUTAG (+4.25%). While slightly trailing the parametric MSGCOT in most cases, it outperforms all other

baselines, demonstrating that multi-scale reasoning drives performance gains. The strong performance of MSGCOT-P highlights the framework’s adaptability to existing graph partitioning strategies.

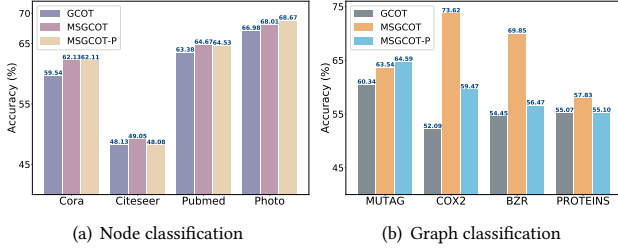


Figure 7: The extensibility analysis of MSGCOT.

5.7 Parameter and Efficiency Analysis

The effectiveness of lightweight coarse-grain networks. The parameters of MSGCOT primarily come from the coarsening network, which consists of two low-rank decomposed matrices. For each layer, the number of parameters can be calculated as $dr + rC^l$. The total parameter count is $Ldr + \sum_{l=1}^L rC^l$. Taking the first layer as an example, we compare the parameter counts with and without the lightweight coarsening network. As shown in Figure 6, node classification nearly reaches its optimal performance at $r = 8$, while graph classification achieves strong results even at $r = 1$. Here, we set $r = 8$ on the node task and $r=1$ on the graph task. As shown in the Table 3, the low-rank design maintains a stable parameter count of 0.43K–4.71K, whereas the full-rank design requires 43.15K–85.17K. The relative ratio is as low as 0.9% (COX2) and up to 6.1% (Cora), validating the efficiency of low-rank decomposition.

Table 3: Comparison of prompt-tuning training parameters on rank and without low rank.

	Cora	Citeseer	MUTAG	COX2
low rank	4.21K	4.71K	0.43K	0.45K
w/o low rank	69.32K	85.17K	43.15K	49.28K
Relative Ratio	6.1%	5.5%	1.0%	0.9%

Trainable prompt parameters. We compare trainable parameters across prompting methods in Table 4. MSGCOT maintains competitive performance while substantially reducing parameters: for node classification (10.37K–21.8K), it achieves a 47.1%–68.3% reduction compared to GCOT (32.76K). In graph classification (1.17K–5.81K), reductions reach 29.1%–85.7% versus GCOT (8.19K), even outperforming GPF+ (5.12K) on MUTAG. This efficiency stems from MSGCOT’s low-rank design ($r=8$ for node tasks, $r=1$ for graph tasks) and task-adaptive coarsening. On average, parameter reductions are 53.6% (node) and 63.2% (graph), highlighting its ability to balance complexity and performance.

Running times. Table 5 compares the per-epoch running times of different prompt tuning methods. MSGCOT shows clear efficiency

Table 4: Comparisons of prompt-tuning training parameters (K) on node and graph classification tasks.

Dataset	GPF+	EdgePrompt+	GCOT	MSGCOT
Cora	5.12	10.24	32.76	10.37
Citeseer	5.12	10.24	32.76	11.01
Pubmed	5.12	10.24	32.76	21.80
Photo	5.12	10.24	32.76	14.81
MUTAG	5.12	10.24	8.19	1.20
BZR	5.12	10.24	8.19	1.17
COX2	5.12	10.24	8.19	1.24
PROTEINS	5.12	10.24	8.19	5.81

benefits across task types. For node classification, its runtime is slightly higher than GPF+ and EdgePrompt+ but comparable to GCOT. For graph classification, MSGCOT averages 0.137 seconds per epoch—a 34.8% reduction over GCOT—while maintaining higher accuracy, attributable to its low coarsening ratio and reduced hidden dimension. Although multi-step prompting adds complexity, MSGCOT’s overhead remains manageable.

Table 5: Comparisons of prompt-tuning times (seconds) on node and graph classification tasks.

Dataset	GPF+	EdgePrompt+	GCOT	MSGCOT
Cora	0.225	0.214	0.246	0.250
Citeseer	0.234	0.214	0.250	0.248
Pubmed	0.224	0.210	0.272	0.282
Photo	0.222	0.213	0.257	0.250
MUTAG	0.182	0.186	0.228	0.127
BZR	0.182	0.192	0.198	0.128
COX2	0.182	0.189	0.201	0.131
PROTEINS	0.185	0.202	0.215	0.161

6 Conclusion

In this paper, we propose MSGCOT, a novel multi-scale prompt chain framework that addresses the limitation of single-granularity prompts in existing graph prompt tuning methods. The proposed low-rank coarsening network captures hierarchical structural features while remaining parameter-efficient. Inspired by the human thought process, MSGCOT progressively introduces information from coarse to fine granularity during prompt generation, leading to more refined prompts. Experimental results demonstrate that MSGCOT consistently outperforms single-granularity approaches and achieves superior performance.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grants 62425605, 62133012, and 62303366, and in part by the Key Research and Development Program of Shaanxi under Grants 2025CY-YBXM-041, 2022ZDLGY01-10, and 2024CY2-GJHX-15.

References

- [1] Karsten M Borgwardt, Cheng Soon Ong, Stefan Schöner, SVN Vishwanathan, Alex J Smola, and Hans-Peter Kriegel. 2005. Protein function prediction via graph kernels. *Bioinformatics* 21, suppl_1 (2005), i47–i56.
- [2] Qin Chen, Liang Wang, Bo Zheng, and Guojie Song. 2025. Dagprompt: Pushing the limits of graph prompting with a distribution-aware graph prompt tuning approach. In *Proceedings of the ACM on Web Conference 2025*. 4346–4358.
- [3] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph neural networks for social recommendation. In *The world wide web conference*. 417–426.
- [4] Taoran Fang, Yunchao Zhang, Yang Yang, Chunping Wang, and Lei Chen. 2023. Universal prompt tuning for graph neural networks. *Advances in Neural Information Processing Systems* 36 (2023), 52464–52489.
- [5] Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. 2023. Towards revealing the mystery behind chain of thought: a theoretical perspective. *Advances in Neural Information Processing Systems* 36 (2023), 70757–70798.
- [6] Xingbo Fu, Yinhan He, and Jundong Li. 2025. Edge prompt tuning for graph neural networks. *arXiv preprint arXiv:2503.00750* (2025).
- [7] Ziqi Gao, Chenran Jiang, Jiawen Zhang, Xiaosen Jiang, Lanqing Li, Peilin Zhao, Huanming Yang, Yong Huang, and Jia Li. 2023. Hierarchical graph learning for protein–protein interaction. *Nature Communications* 14, 1 (2023), 1093.
- [8] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang, and Jie Tang. 2022. Graphmae: Self-supervised masked graph autoencoders. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 594–604.
- [9] George Karypis and Vipin Kumar. 1997. METIS: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices. (1997).
- [10] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv:1609.02907* (2016).
- [11] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [12] Junhyun Lee, Woosong Yang, and Jaewoo Kang. 2024. Subgraph-level universal prompt tuning. *arXiv preprint arXiv:2402.10380* (2024).
- [13] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys* 55, 9 (2023), 1–35.
- [14] Zemin Liu, Xingtong Yu, Yuan Fang, and Xinming Zhang. 2023. Graphprompt: Unifying pre-training and downstream tasks for graph neural networks. In *Proceedings of the ACM web conference 2023*. 417–428.
- [15] Yuanfu Lu, Xunqiang Jiang, Yuan Fang, and Chuan Shi. 2021. Learning to pre-train graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 4276–4284.
- [16] Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z Sheng, Hui Xiong, and Leman Akoglu. 2021. A comprehensive survey on graph anomaly detection with deep learning. *IEEE transactions on knowledge and data engineering* 35, 12 (2021), 12012–12038.
- [17] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the construction of internet portals with machine learning. *Information Retrieval* 3, 2 (2000), 127–163.
- [18] Seth A Myers, Aneesh Sharma, Pankaj Gupta, and Jimmy Lin. 2014. Information network or social network? The structure of the Twitter follow graph. In *Proceedings of the 23rd international conference on world wide web*. 493–498.
- [19] Andrew Ng, Michael Jordan, and Yair Weiss. 2001. On spectral clustering: Analysis and an algorithm. *Advances in neural information processing systems* 14 (2001).
- [20] Ryan Rossi and Nesreen Ahmed. 2015. The network data repository with interactive graph analytics and visualization. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 29.
- [21] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI magazine* 29, 3 (2008), 93–93.
- [22] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [23] Fan-Yun Sun, Jordan Hoffmann, Vikas Verma, and Jian Tang. 2019. Infograph: Un-supervised and semi-supervised graph-level representation learning via mutual information maximization. *arXiv preprint arXiv:1908.01000* (2019).
- [24] Mingchen Sun, Kaixiong Zhou, Xin He, Ying Wang, and Xin Wang. 2022. Gppt: Graph pre-training and prompt tuning to generalize graph neural networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1717–1727.
- [25] Xiangguo Sun, Hong Cheng, Jia Li, Bo Liu, and Jihong Guan. 2023. All in one: Multi-task prompting for graph neural networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2120–2131.
- [26] Xiangguo Sun, Jiawen Zhang, Xixi Wu, Hong Cheng, Yun Xiong, and Jia Li. 2023. Graph prompt learning: A comprehensive survey and beyond. *arXiv preprint arXiv:2311.16534* (2023).
- [27] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv:1710.10903* (2017).
- [28] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2018. Deep graph infomax. *arXiv preprint arXiv:1809.10341* (2018).
- [29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [30] Wei Xu, Xin Liu, and Yihong Gong. 2003. Document clustering based on non-negative matrix factorization. In *Proceedings of the 26th annual international ACM SIGIR conference on Research and development in information retrieval*. 267–273.
- [31] Yaming Yang, Ziyu Guan, Zhe Wang, Wei Zhao, Cai Xu, Weigang Lu, and Jianbin Huang. 2022. Self-supervised heterogeneous graph pre-training based on structural clustering. *Advances in Neural Information Processing Systems* 35 (2022), 16962–16974.
- [32] Zhitao Ying, Jiaxuan You, Christopher Morris, Xiang Ren, Will Hamilton, and Jure Leskovec. 2018. Hierarchical graph representation learning with differentiable pooling. *Advances in neural information processing systems* 31 (2018).
- [33] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.
- [34] Xingtong Yu, Jie Zhang, Yuan Fang, and Renhe Jiang. 2024. Non-homophilic graph pre-training and prompt learning. *arXiv preprint arXiv:2408.12594* (2024).
- [35] Xingtong Yu, Chang Zhou, Zhongwei Kuai, Xinming Zhang, and Yuan Fang. 2025. GCoT: Chain-of-thought prompt learning for graphs. *arXiv preprint arXiv:2502.08092* (2025).
- [36] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. *Advances in neural information processing systems* 31 (2018).
- [37] Ziyu Zheng, Yaming Yang, Ziyu Guan, Wei Zhao, and Weigang Lu. 2025. Discrepancy-Aware Graph Mask Auto-Encoder. *arXiv preprint arXiv:2506.19343* (2025).
- [38] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. 2022. Conditional prompt learning for vision-language models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 16816–16825.

A Experiment Setting

A.1 Dataset Details

Table 6: Dataset statistics

Datasets	Graphs	Nodes	Edges	Feats	Classes	Task* (N/G)
Cora	1	2,708	5,429	1,433	7	N
Citeseer	1	3,327	4,732	3,703	6	N
Pubmed	1	19,717	88,648	500	3	N
Photo	1	7,650	238,162	745	8	N
MUTAG	188	17.9	18.9	7	2	G
COX2	467	41.2	43.5	3	2	G
BZR	405	35.8	38.4	3	2	G
PROTEINS	1,113	39.1	72.8	4	2	G

A.2 Hyperparameter Setting

As shown in the Table 7, we provide detailed parameter configurations for each dataset. cr represents the coarsening rate, cl represents the number of coarsening layers, and r represents the hidden layer dimension of the low-rank coarsening network.

Table 7: Hyperparameter settings on different datasets.

Datasets	c_r	c_l	r	lr	wd	encoder_layer
Cora	0.1	2	8	1e-3	0	2
Citeseer	0.1	2	8	1e-3	0	2
Pubmed	0.1	1	8	1e-3	0	3
Photo	0.3	2	8	1e-3	0	2
MUTAG	0.05	2	1	1e-3	1e-4	3
COX2	0.01	2	1	1e-3	1e-4	3
BZR	0.01	2	1	1e-3	1e-4	3
PROTEINS	0.1	2	1	1e-3	1e-4	3

B More Experiments

B.1 Effectiveness on different pre-training strategies

To confirm the generalizability of our method across different pre-training strategies, we employ a GraphCL-pretrained encoder and compare it with several state-of-the-art prompt learning methods, all of which are pre-training-agnostic. As shown in the Table 8 and Table 9, MSGCOT achieves the best performance on 7 out of 8 benchmark datasets, with particularly significant improvements of +18.23% and +25.63% in absolute accuracy on biochemical datasets (COX2 and BZR), respectively. The results indicate that our proposed multi-scale graph chain-of-thought framework does not rely on specific pre-training objectives, validating MSGCOT's potential as a universal prompting framework.

Table 8: Performance comparison on node classification tasks. The best and second-best results are highlighted in bold and underlined, respectively.

	Cora	Citeseer	Pubmed	Photo
GPF+	62.47±14.24	49.10±9.27	54.75±10.56	57.99±10.13
EdgePrompt+	63.08±13.37	50.53±9.15	56.17±9.93	57.97±10.19
GCOT	<u>63.68±13.79</u>	50.00±10.20	<u>56.42±10.41</u>	60.41±10.82
MSGCOT	64.24±17.20	50.56±9.46	56.69±12.92	<u>58.45±10.80</u>

Table 9: Performance comparison on graph classification tasks. The best and second-best results are highlighted in bold and underlined, respectively.

	MUTAG	COX2	BZR	PROTEINS
GPF+	55.12±15.79	56.35±16.57	<u>49.63±15.17</u>	56.06±9.49
EdgePrompt+	55.60±14.41	<u>56.46±16.37</u>	49.44±14.91	<u>56.33±9.60</u>
GCOT	<u>55.75±14.35</u>	52.37±15.51	49.61±8.48	55.12±10.54
MSGCOT	56.42±15.09	74.58±4.16	75.26±4.74	59.02±6.58

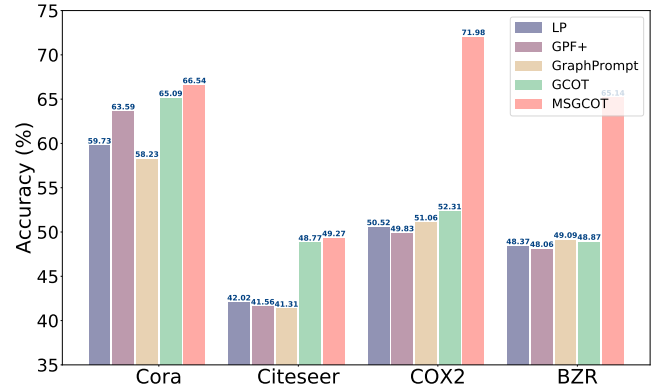


Figure 8: The effectiveness of GAT encoders of MSGCOT.

B.2 Effectiveness on different graph encoders

To validate the effectiveness of MSGCOT across different encoders, we replaced the original GCN encoder with a graph attention encoder and conducted one-shot node classification experiments on the node classification datasets Cora and Citeseer, alongside the graph classification datasets COX2 and BZR. We present the mean across 100 trials. As shown in Figure 9, our approach maintains optimal performance across various datasets under the GAT configuration. This demonstrates that our method constitutes a generalised graph prompt approach whose efficacy remains independent of specific graph encoder architectures.

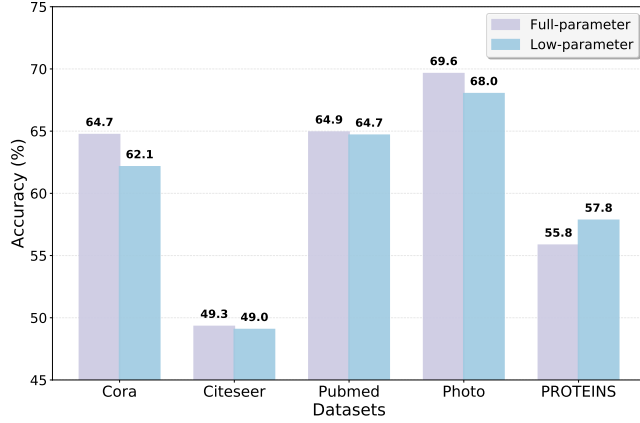


Figure 9: Performance comparison between low-rank and full-parameter coarsening networks.

Table 12: Performance comparison of Top-K prompt filtering strategies

Top-K	Cora	Citeseer	PROTEINS
3	59.77 ± 16.43	48.55 ± 10.52	54.59 ± 6.51
5	59.89 ± 15.66	48.60 ± 10.61	56.49 ± 7.34
10	60.20 ± 15.97	48.34 ± 10.79	56.33 ± 7.45

B.3 The Performance on Heterophilous Graph Datasets

Table 10: Performance comparison on heterophilic node classification tasks. The best and second-best results are highlighted in bold and underlined, respectively.

	Texas	Cornell	Wisconsin	Chameleon
LP	30.26±16.19	22.62±6.04	24.29±6.16	26.61±5.89
GraphPrompt	27.83±13.92	22.38±6.15	23.97±5.86	25.93±6.21
GPF+	26.46±14.80	<u>24.07±8.96</u>	24.12±7.97	26.36±4.88
GCOT	<u>29.37±14.76</u>	<u>24.16±6.88</u>	<u>24.00±6.02</u>	<u>27.01±5.62</u>
MSGCOT	30.29±15.42	24.68±6.82	25.25±7.39	27.27±6.29

To evaluate the effectiveness of MSGCOT in heterophilous graph scenarios, we conducted experiments across multiple distinct heterophilous graph datasets, maintaining consistent experimental settings with previous work. Compared to existing methods, we achieved state-of-the-art performance on four datasets—Texas, Wisconsin, Cornell, and Chameleon, validating the efficacy of our multi-scale graph prompt mechanism. These results demonstrate that our multi-scale prompting approach is compatible with both homophilous and heterophilous graphs.

B.4 Serial vs. Parallel Prompt Fusion Strategies

We compared the performance of serial (coarse-to-fine) and parallel multi-scale prompt fusion strategies. Experimental results show that the serial strategy consistently outperforms the parallel strategy across multiple datasets, validating the effectiveness of the coarse-to-fine reasoning process.

Table 11: Performance comparison between serial and parallel prompt fusion strategies

Strategy	Cora	Citeseer	PROTEINS
Parallel	59.85 ± 15.97	45.07 ± 10.79	54.52 ± 8.91
Serial	62.13 ± 17.53	49.05 ± 11.41	57.83 ± 2.71

B.5 Low-Rank vs. Full-Parameter Coarsening Network

We compared the performance of the low-rank designed coarsening network with a full-parameter version. The low-rank design significantly outperforms the full-parameter version in parameter efficiency while maintaining competitive performance.

B.6 Prompt Abalation Experiments

Prompt Filtering (Top-K). We tested a prompt selection strategy based on Top-K sampling. Experiments show that selective prompting improves efficiency while maintaining performance.

Multi-Layer Prompt Concatenation. We concatenate prompts from all layers as the final prompt. Experimental results show that this strategy does not lead to significant performance improvement, indicating that subsequent layers in the chain-of-thought reasoning naturally incorporate information from previous layers.

Table 13: Performance comparison between multi-layer prompt concatenation and the original method

Method	Citeseer	Photo	PROTEINS
Concatenation	48.54 ± 9.86	67.99 ± 10.25	58.43 ± 16.26
Original	49.05 ± 11.41	68.01 ± 10.93	57.22 ± 7.03

B.7 Large-Scale Dataset Experiment

We conducted extended experiments on the ogbn-arxiv dataset. MSGCOT demonstrates good scalability.

Table 14: Performance comparison on the ogbn-arxiv dataset

Method	ogbn-arxiv
GPrompt	17.74 ± 3.61
GCOT	18.85 ± 3.66
MSGCOT	20.66 ± 3.90

B.8 Graph Classification on Social Network

We supplemented graph classification experiments on the IMDB-BINARY and IMDB-MULTI datasets, further validating the effectiveness of the method on diverse graph structures.

Table 15: Performance comparison on social network graph classification datasets

Dataset	GPF+	GPrompt	MSGCOT
IMDB-BINARY	50.83 ± 6.58	52.21 ± 5.34	54.21 ± 6.89
IMDB-MULTI	33.65 ± 5.23	34.59 ± 2.90	35.30 ± 4.01

B.9 The Performance on Different Pooling Method

We compared the performance using DiffPool as the coarsening mechanism. Experiments show that directly using pre-trained representations combined with the low-rank coarsening network avoids overfitting caused by excessive compression, yielding better performance.

Table 16: Comparison experiment with DiffPool

Method	Cora	MUTAG
MSGCOT (DiffPool)	48.83 ± 12.33	35.31 ± 7.90
MSGCOT (Original)	62.13 ± 17.53	63.54 ± 14.94